

Dataset Selection

We chose the poker dataset with the goal of making a model that can correctly identify poker hands.

EDA

No missing values; the data itself is already pretty 'clean' by default as it was intentionally crafted instead of collected data.

Data Cleaning

No data cleaning needed

Model Selection

Null Model: set everything as majority class

Simple model: KNN

Complex model: Neural Net

Model Tuning Documentation/Notes

Note: While testing and fine-tuning, each iteration was taking hours, so we decided to start documenting in case we don't get results that are as good as we would want them to be. .

KNN → Decision Tree

- Grid search
- Changed from KNN to Decision Tree as we realized KNN is not great with huge datasets due to the curse of dimensionality.

Neural Net - MLPClassifier

- Tried with and without StandardScaler
 - Accuracy was worse without it
- Changed solver from 'lgfps' to 'adam'
 - Increased overall execution time

- Tinkered with different alpha's
 - Best alpha we found for our use case was $1e-5$
- Tested various different amount of hidden layers and eventually included neurons after 2 hours of testing 😭
 - Best solution currently for hidden_layer_sizes = (12,24)
- Changed max_iter to 1000 since old solver needed more iterations
- We tried using various activation functions.
 - Tried Logistics regression, but it performed poorly
 - Best function so far: Tanh
- We started running the model locally on our PCs to see if it was faster.
 - It didn't work on Ando's but we saw a dramatic improvement using Jordan's PC. Even then, each run still took between 10-20 ()
- We included a line that prevents overfitting and is supposed to reduce training time by stopping when validation score stops improving
 - early_stopping = true
- We decided to run one big batch of GridSearch with the parameters below and let it sit for the night.

```

pipe = Pipeline(steps=[('classifier', 'passthrough')])

seed=42
param_grid = [
    { # Null Model
      'classifier': [DummyClassifier(strategy='most_frequent',
random_state=seed)]
    },
    { # KNN
      'classifier': [KNeighborsClassifier()],
      'classifier__n_neighbors': [50, 120, 500, 1000]
    },
    { # Neural Net
      'classifier': [MLPClassifier(random_state=seed, max_iter = 1000,
early_stopping = True)],
      'classifier__hidden_layer_sizes': [(12,24), (12,25), (12,16)],
      'classifier__activation': ['relu', 'tanh', 'logistic'],
      'classifier__solver': ['adam', 'sgd'],
      'classifier__alpha': [1e-5, 1e-6, 1e-7],
      'classifier__learning_rate': ['constant', 'adaptive'],
      'classifier__learning_rate_init': [1e-4, 1e-3, 1e-2, 1]
    }
]

```

- We ran it through the night in colab and locally.
 - Our colab workbook ran into a runtime error since it took too long, and the local run was not finished yet. We decided to remove some settings to reduce runtime.
- We found out that that colab has a runtime type that is adapted for Machine Learning. It greatly cut runtime so we decided to experiment with greater numbers of hidden layers.
 - We achieved an accuracy of 99.8% which is as good as it gets.
- While we tried various hyperparameters using grid search, for the final code, we cut down the number of hyperparameters in the GridSearch to the best ones we had to keep things quick.